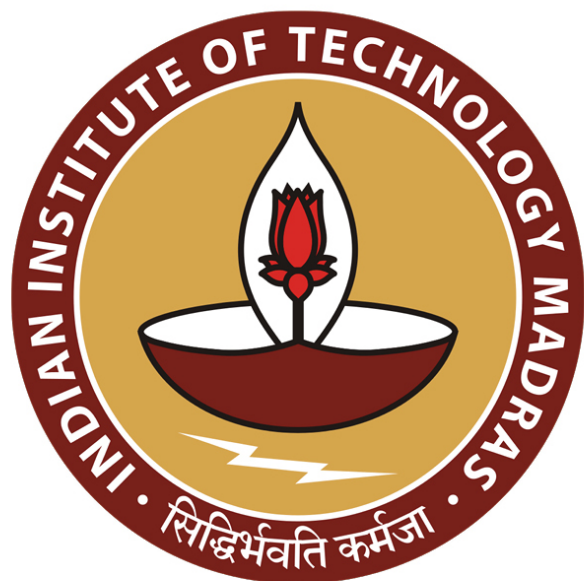


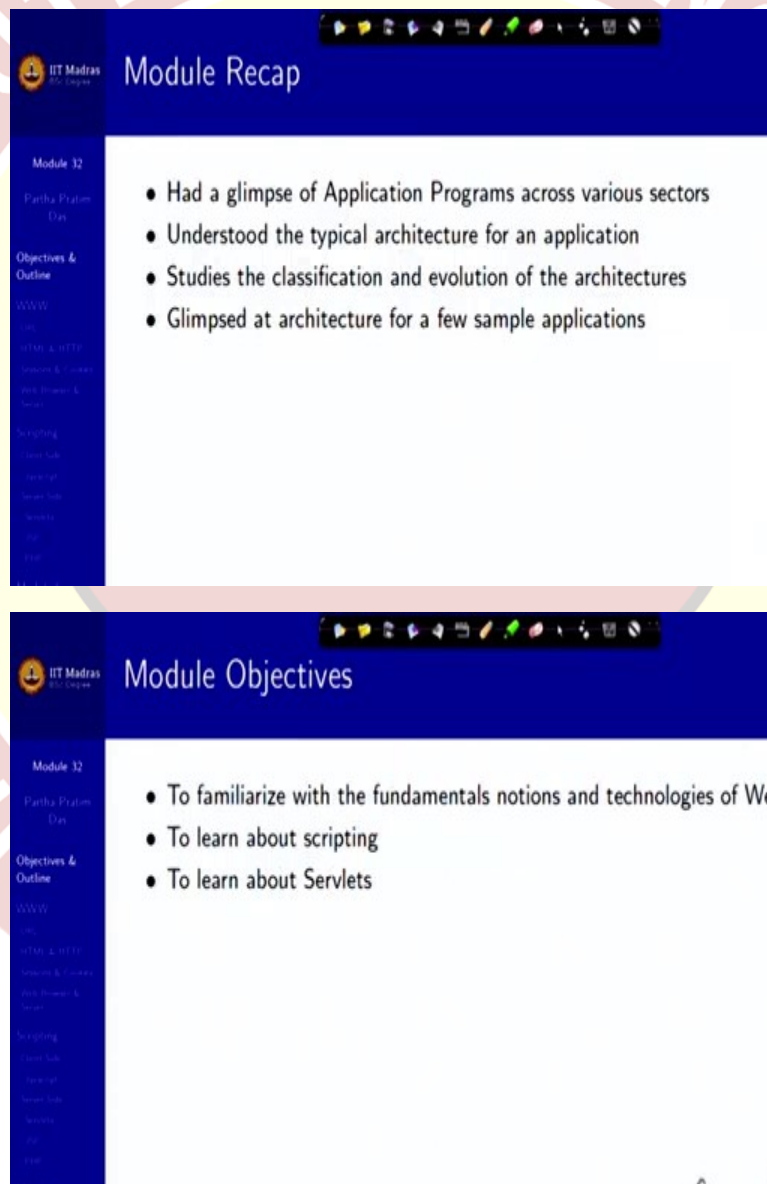


IIT Madras
ONLINE DEGREE

Database Management Systems
Professor Partha Pratim Das
Department of Computer Science & Engineering,
Indian Institute of Technology Kharagpur
Application Design and Development/2: Web Applications

Welcome to Module 32 of Database Management Systems course in the IIT Madras online B.Sc. program.

(Refer Slide Time: 00:24)

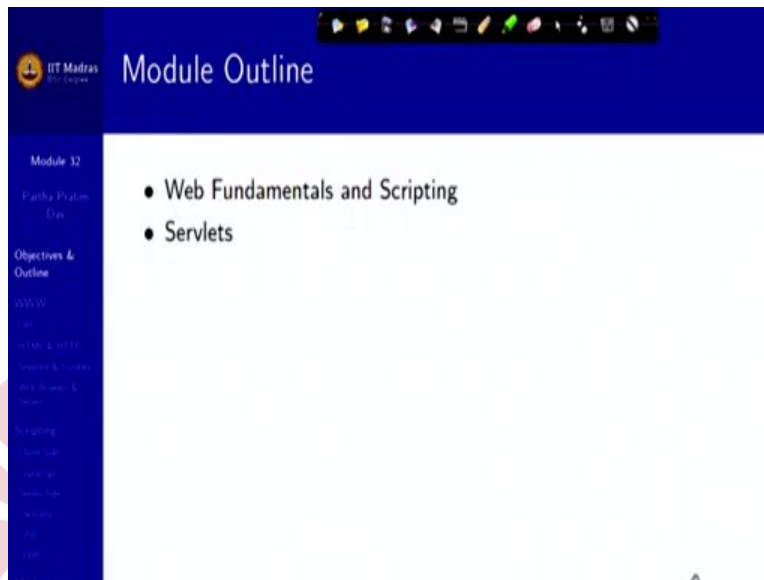


Module Recap

- Had a glimpse of Application Programs across various sectors
- Understood the typical architecture for an application
- Studies the classification and evolution of the architectures
- Glimpsed at architecture for a few sample applications

Module Objectives

- To familiarize with the fundamentals notions and technologies of Web
- To learn about scripting
- To learn about Servlets



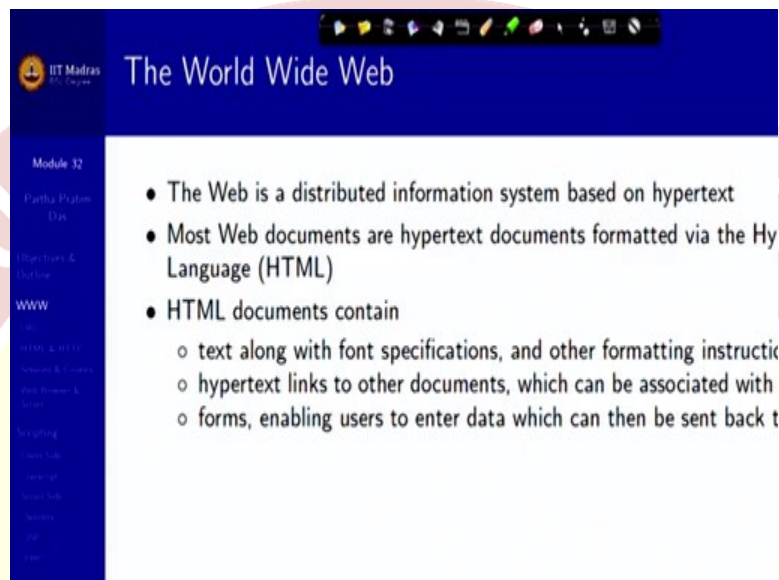
In the last module we have started discussing about how to develop applications using databases at the backend. We had a glimpse of a collection of application programs. We understood that in spite of the variety there is a strong commonality which we call the architecture of these applications, which is typically three-tier presentation, business logic and data access layer. We saw the classification and evolution of architectures and taken a glimpse at few sample applications which you are to study and understand in more detail yourself. So, this is what we are going to do.

(Refer Slide Time: 01:11)



I will start with, though it is not really a part of this course, but since I am not sure how much, how many of you are familiar, so I will start with some fundamental definitions of the World Wide Web, www, so web fundamentals.

(Refer Slide Time: 01:30)



The screenshot shows a presentation slide titled "The World Wide Web" from IIT Madras. The slide is part of Module 32, titled "Partha Pratim Das". The slide content is as follows:

- The Web is a distributed information system based on hypertext
- Most Web documents are hypertext documents formatted via the Hypertext Markup Language (HTML)
- HTML documents contain
 - text along with font specifications, and other formatting instructions
 - hypertext links to other documents, which can be associated with certain regions of the text
 - forms, enabling users to enter data which can then be sent back to the server

So, it is a distributed information system, which is based on hypertext, which means that most documents are hypertext document. And they are formatted using HTML which is Hypertext Markup Language, which is a W3C standard. So, that contains texts along with specification of the font and other formatting. And it has, why is it hypertext, because it has links to other documents, which is associated, it can be associated with certain regions of the text is a click this URL. It can contain forms to collect data from the user. So, this is, this basically is our basic document definition.

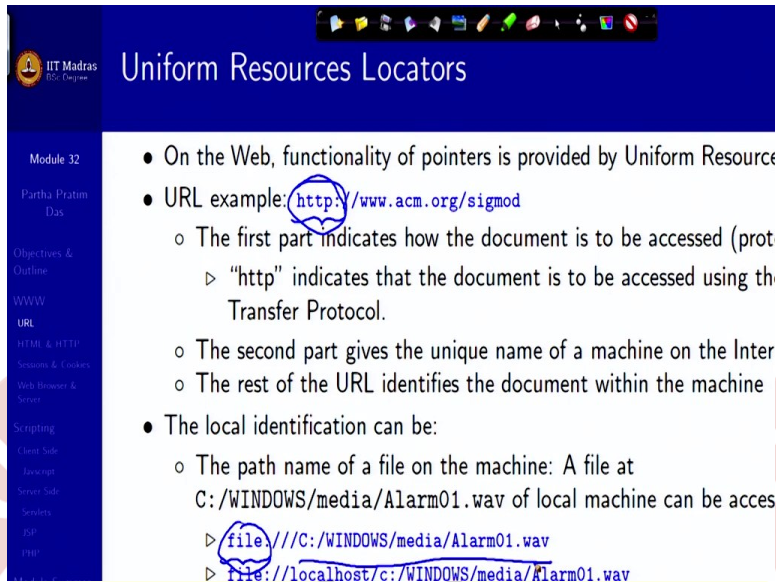
(Refer Slide Time: 02:20)

Uniform Resources Locators

- On the Web, functionality of pointers is provided by Uniform Resource
- URL example: <http://www.acm.org/sigmod>
 - The first part indicates how the document is to be accessed (protocol)
 - ▷ "http" indicates that the document is to be accessed using the Transfer Protocol.
 - The second part gives the unique name of a machine on the Internet
 - The rest of the URL identifies the document within the machine
- The local identification can be:
 - The path name of a file on the machine: A file at C:/WINDOWS/media/Alarm01.wav of local machine can be accessed
 - ▷ <file:///C:/WINDOWS/media/Alarm01.wav>
 - ▷ <file://localhost/c:/WINDOWS/media/Alarm01.wav>

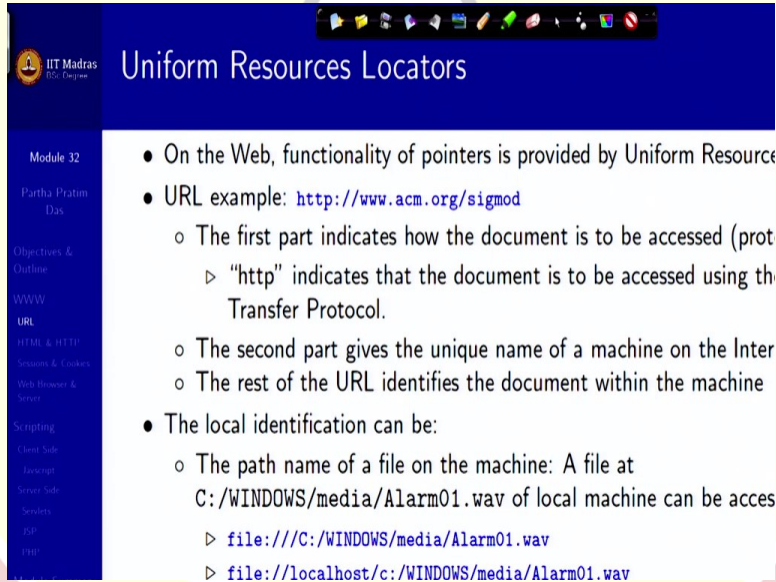
Uniform Resources Locators

- On the Web, functionality of pointers is provided by Uniform Resource
- URL example: <http://www.acm.org/sigmod>
 - The first part indicates how the document is to be accessed (protocol)
 - ▷ "http" indicates that the document is to be accessed using the Transfer Protocol.
 - The second part gives the unique name of a machine on the Internet
 - The rest of the URL identifies the document within the machine
- The local identification can be:
 - The path name of a file on the machine: A file at C:/WINDOWS/media/Alarm01.wav of local machine can be accessed
 - ▷ <file:///C:/WINDOWS/media/Alarm01.wav>
 - ▷ <file://localhost/c:/WINDOWS/media/Alarm01.wav>



Uniform Resources Locators

- On the Web, functionality of pointers is provided by Uniform Resource
- URL example: <http://www.acm.org/sigmod>
 - The first part indicates how the document is to be accessed (protocol)
 - ▷ "http" indicates that the document is to be accessed using the Transfer Protocol.
 - The second part gives the unique name of a machine on the Internet
 - The rest of the URL identifies the document within the machine
- The local identification can be:
 - The path name of a file on the machine: A file at C:/WINDOWS/media/Alarm01.wav of local machine can be accessed
 - ▷ <file:///C:/WINDOWS/media/Alarm01.wav>
 - ▷ <file://localhost/c:/WINDOWS/media/Alarm01.wav>



Uniform Resources Locators

- On the Web, functionality of pointers is provided by Uniform Resource
- URL example: <http://www.acm.org/sigmod>
 - The first part indicates how the document is to be accessed (protocol)
 - ▷ "http" indicates that the document is to be accessed using the Transfer Protocol.
 - The second part gives the unique name of a machine on the Internet
 - The rest of the URL identifies the document within the machine
- The local identification can be:
 - The path name of a file on the machine: A file at C:/WINDOWS/media/Alarm01.wav of local machine can be accessed
 - ▷ <file:///C:/WINDOWS/media/Alarm01.wav>
 - ▷ <file://localhost/c:/WINDOWS/media/Alarm01.wav>

Uniform Resources Locators

- On the Web, functionality of pointers is provided by Uniform Resource
- URL example: <http://www.acm.org/sigmod>
 - The first part indicates how the document is to be accessed (protocol)
 - ▷ "http" indicates that the document is to be accessed using the Transfer Protocol.
 - The second part gives the unique name of a machine on the Internet
 - The rest of the URL identifies the document within the machine
- The local identification can be:
 - The path name of a file on the machine: A file at C:/WINDOWS/media/Alarm01.wav of local machine can be accessed
 - ▷ <file:///C:/WINDOWS/media/Alarm01.wav>
 - ▷ <file://localhost/c:/WINDOWS/media/Alarm01.wav>

Now, for identity, we have a what is called a uniform resource locator. That is, since documents are distributed worldwide, I need to have a mechanism to identify it locate it. So, we have a URL. And if you look at this URL, there is a initial part which is http standing for hypertext transfer protocol, which says in what mechanism I should interact with that document. And we will talk a little bit more about that later.

Second part locates the unique name of the machine in the Internet in this entire world which is supposed to have that document and whatever you have after that could be very, very long is the actual location of the document within that machine. So, one is how you interact with the document, where do you find the document, which machine and then the rest is the location in the machine. So, it could be also a document in your machine or in some other machine. So, for example, if I want to access this document, a local machine, then I will use a starting which is file colon, different from http colon. This tells us go and look in the Internet, world wide web. This tells us look in the local machine. And then it gives the path.

So, since this pertains to local machine, so I do not need to say that how to identify the machine. I can just directly go to the path of the document. Of course, there is an alternate way. The local machine can be called a local host, which could be, which could again have an IP address, which is locating the machine and then you have the path to the actual document in that machine. Besides the protocol, the machine, you could also actually send a query to that. Instead of looking for a document you can send a query. So, you can send parameters with this request and

that is, here, what we are saying is you do search with parameter q is equal to silberschatz. So, you can give multiple parameters. So, that is the basic purpose of the URL.

(Refer Slide Time: 05:15)

The image shows two identical slides from IIT Madras, titled "URI, URL, and URN". The slides are part of Module 32, presented by Partha Pratim Das. The content is as follows:

- Uniform Resource Identifier (URI)
- Uniform Resource Locator (URL)
- Uniform Resource Name (URN)
- Relationships:
 - URIs can be classified as locators (URLs), or as names (URNs), or as both.
 - URN functions like a person's name
 - URL resembles that person's street address.
 - URN defines an item's identity, while the URL provides a method for finding it.

On the right side of each slide, there is a Venn diagram illustrating the relationship between these three concepts:

- A large blue circle represents the **URI**.
- A red circle represents the **URN**, which is entirely contained within the URI circle.
- A green circle represents the **URL**, which overlaps with the URN circle.
- Inside the URN circle, there are two examples: `dmn.tld/page.htm` and `ste.org/img.png`.
- Inside the URL circle, there are two examples: `https://` and `ftp://st`.
- Below the URN circle, there is a small box labeled `data.htm`.

And typically you will hear that there are three terms that go around. Of course, URL is something which you are most familiar with, which is this whole thing of this. This is called URN that is the name of the document. This is the, I am sorry, this is URI, the identifier, the actual what name the document is stored with. This is, these are called the URN, Uniform Resource Name. So, URN is more like a person's name and URL is more like person's address

where you can find that person. So, we will typically use the term URL and at times when we want to generalize we will say URI, but mostly URL.

(Refer Slide Time: 06:18)

HTML and HTTP

- HTML provides formatting, hypertext link, and image display features:
 - including tables, stylesheets (to alter default formatting), etc.
- HTML also provides input features
 - Select from a set of options
 - ▷ Pop-up menus, radio buttons, check lists
 - Enter values
 - ▷ Text boxes
 - Filled in input sent back to the server, to be acted upon by an external server
- HyperText Transfer Protocol (HTTP) used for communication with the server

HTML we have talked off. HTTP is a protocol which is used for communicating with the machine which has or which knows where your document can be found. HTML, as we said, will select from a set of options, pop-up menus, radio, etc., will enter values and you can fill up the input as you always do.

(Refer Slide Time: 06:49)

Sample HTML

Search for:

```
<html>
<body>
  <table border=1>
    <tr>
      <th>ID</th>
      <th>Name</th>
      <th>Dep.
    </tr>
    <tr>
      <td>00128</td>
      <td>Zhang</td>
      <td>Comp. Sci.
    </tr>
    <tr>
      <td>12345</td>
      <td>Shankar</td>
      <td>Comp. Sci.
    </tr>
    <tr>
      <td>19991</td>
      <td>Brandt</td>
      <td>History
    </tr>
  </table>
  <form action="/PersonQuery" method="get">
    Search for:
    <select name="persontype">
      <option value="student" selected>Student</option>
      <option value="instructor">Instructor</option>
    </select>
    <br>
    Name: <input type="text" size=20 name="name">
    <input type="submit" value="Submit" />
  </form>
</body>
</html>
```

This is an HTML sample just for your understanding. So, HTML is these are known as tags. So, you have a corner bracket and you write some, these are predefined. I mean, in certain way you can define them, but take this as predefined. Every tag has a meaning. So, it is a parenthesized expression, which starts from a tag HTML and ends with the same tag and the end is marked by having a forward slash before the tag name.

So, I have the whole HTML page. And in this I have a body which gives me the body of the page. In the body I have a table and the tag actually is table and the tag qualifier is border which says that the table must come with a border. This is the end of table. And then we have tags for table entries. So, tr is table row. This table row ends. So, we are talking about this, first row th says it is a table header. So, you can see that it is, it comes as a different font. It comes in bold and little bigger.

So, th ID th tells you that ID is a header. This is the next header. This is the next header. So, you can see there are three headers. There is the first row. This is the second row which says table data td, the next one, 00128, then Zhang, then comp sci the three. So, everything is in terms of parenthesis and this entire thing forms another row and the sequence of rows any number of them will be the entire table as you can see.

So, after that you have form action. So, you have another HTML element called form which is to enter data. Now, we say with the form we say the qualifier is action for which we have given a name person query. How do we know this action? So, we will call it person query. Method is get which says that when the HTML is completed, submitted, then what should we do you should get that data. That is what I intend to do.

Now, this on this form, here if you look at, then I have written search for which is a normal text comes as search for and then here I have a drop down which has multiple options. One option is student, one option is instructor. So, it is saying option begin option end, option begin option end. So, as I click on here at this point, this will drop down and whatever I have selected, it gets becomes the option that have taken.

Then I have a break, because I want to go to the next line. Then I was plain text name, which I say is followed by a box, edit box, input type is text, size is 20, its internal name is name. So, I will identify whatever you enter. I will identify it as name. And then I have input type is submit,

the value is submit that it will, you will submit this to get value is submit. So, this button shows as a submit. So, if you just correspond you will be able to, and after you have done this for a couple of HTML pages and corresponding displays, you will learn this pretty easily. It is not that you regularly need to write in raw HTML, but it helps to be able to read it quickly.

(Refer Slide Time: 11:23)

The screenshot shows a presentation slide titled "HTTP and Sessions" from IIT Madras. The slide content is as follows:

- The HTTP protocol is **connectionless**
 - That is, once the server replies to a request, the server closes the the client, and forgets all about the request
 - In contrast, Unix logins, and JDBC/ODBC connections stay conn client disconnects
 - ▷ retaining user authentication and other information
 - Motivation: reduces load on server
 - ▷ operating systems have tight limits on number of open connect
- Information services need session information
 - For example, **user** authentication should be done only once per se
- Solution: use a **cookie**

Now, HTTP protocol is connectionless. What is connectionless? Connectionless is say you have, when you call someone on a phone, then it is connection oriented, because you dial, you get connected as long as you want to talk, and then you disconnect. So, you are thoroughly connected. But when you send a WhatsApp message, it is connectionless, because you send a message, the person will see it at some point of time and the person will respond. You will receive that message and you will see at some point of time. It is not necessary that both of you have to be present at the same time and remain connected all through for this communication to go on. So, that is connectionless.

So, HTTP is connectionless. You send a request, you get a response from the servers. You send another request, you get a response. Now, this is fine as long as you are just requesting for static pages like in Google. You put a search query, you sent, it gives you a response. You read that, go to the links and so on. But suppose when you are in net banking, you have logged in, send the request, you have got the login, now when you send the next request that show me the account

details, the server has to know that it is the same you who were authenticated earlier, who is asking for that report, otherwise it cannot be given.

Which means that every time this kind of an authentication information has to go with your request for information from the server. This becomes a huge load on the server. Also becomes a pretty much of a security risk, because your sending your authentication information every time. So, if somebody can, somebody, everybody will get a lot more of opportunity to break it. So, we use something here as what is called a cookie. Cookie is a small unit of information.

So, when you do the authentication, send the request you are authenticated, the response comes back saying that you are authenticated and it says it starts a session has started. That is for a certain period of time as you keep on doing the interaction the server is going to remember that it is you. How will the server do that? It will send you a piece of information called a cookie.

(Refer Slide Time: 14:21)



Sessions and Cookies

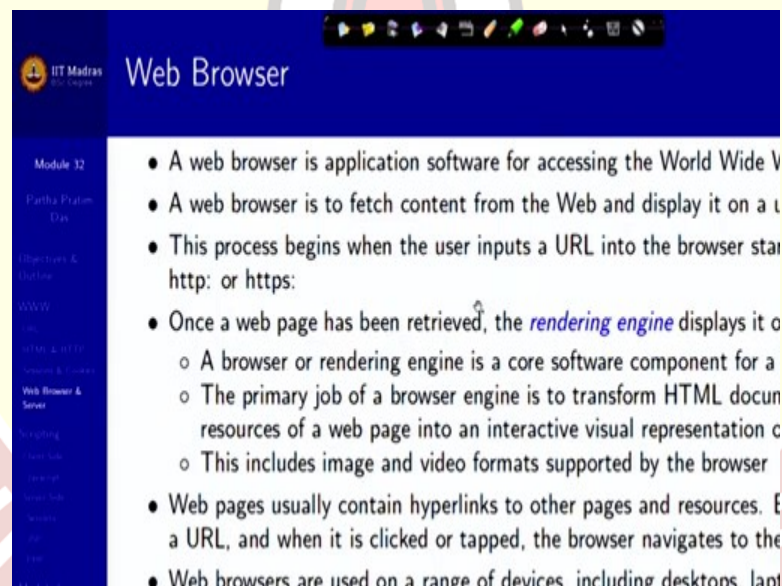
- A **cookie** is a small piece of text containing identifying information
 - Sent by server to browser
 - ▷ Sent on first interaction, to identify session
 - Sent by browser to the server that created the cookie on further i
 - ▷ part of the HTTP protocol
 - Server saves information about cookies it issued, and can use it w request
- Cookies can be stored permanently or for a limited time

The small text contains identifying information in encrypted form. Now, every time the time you make the next request, this cookie goes back with that request and server as it has issued the cookie remembers that it has issued the cookie. So, when it gets the request along with a cookie, it checks whether that that is a valid cookie which is live scale. If it then it continues the operation. But if it does not find a cookie here, then it sends for fresh authentication. So that is the basic mechanism. And for the time for which this liveliness of the, I should say, virtual connection, this is not direct connection, but a virtual connection remains is called a session.

So, while working on different net banking or booking sites and so on, you must have come across that you have 10 seconds left, you have two minutes left, and then you have suddenly told your connection will get timeout. That is the concept of the timeout. Now, the question is, why does not the server remember the cookie forever? That is for several reasons. One is in that way, it does not, if you forget to explicitly log out and cancel the cookie, it will never get lost. It will remain to be in the server and server will create a lot of this chunk.

But a more important concern is it may be possible that you are not using the application for a long time means that you may not have access to that application. Somebody else may have got an access to the application. So, to protect you, it chooses a timeout option that after this much time from the last request if no other request has happened, then it is timed out. The cookie is considered dirty and thrown away. So, that is the concept of a session.

(Refer Slide Time: 16:32)

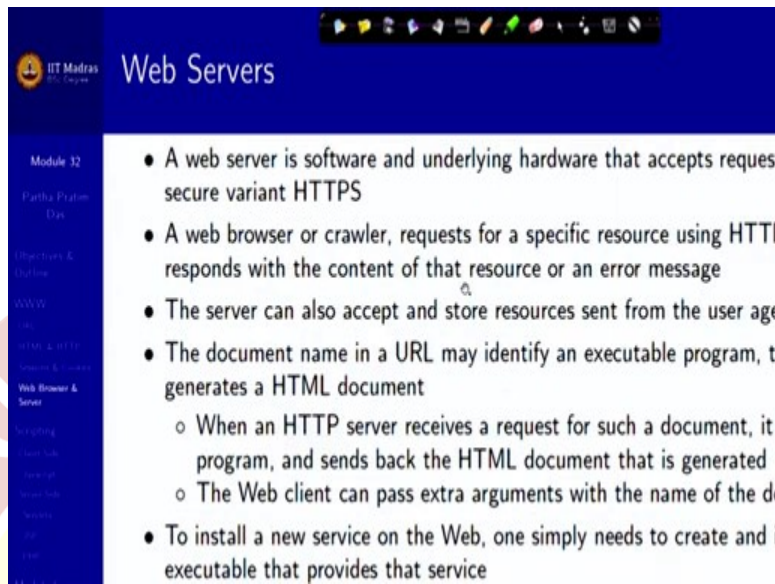


The screenshot shows a presentation slide titled "Web Browser" from IIT Madras. The slide is part of "Module 12" and lists several bullet points:

- A web browser is application software for accessing the World Wide Web.
- A web browser is to fetch content from the Web and display it on a user's device.
- This process begins when the user inputs a URL into the browser starting with http: or https:.
- Once a web page has been retrieved, the **rendering engine** displays it on the screen.
 - A browser or rendering engine is a core software component for a web browser.
 - The primary job of a browser engine is to transform HTML document resources of a web page into an interactive visual representation on the screen.
 - This includes image and video formats supported by the browser.
- Web pages usually contain hyperlinks to other pages and resources. When a user clicks on a URL, and when it is clicked or tapped, the browser navigates to the resource.
- Web browsers are used on a range of devices, including desktops, laptops, and mobile devices.

Now, web browser, all of you know. The web browser is the application program through which HTML is rendered. The rendering is a process by which the text is made into the graphics. And there are certain types of styling and non-text elements can be shown like images, videos, even sound. So, there is a rendering machine which does that. I will not take more time to go into the browsers. As of last year, it is estimated that about nearly 5 billion people used a browser, most used is Google Chrome with 64%, followed by Apple's Safari at 19%.

(Refer Slide Time: 17:17)

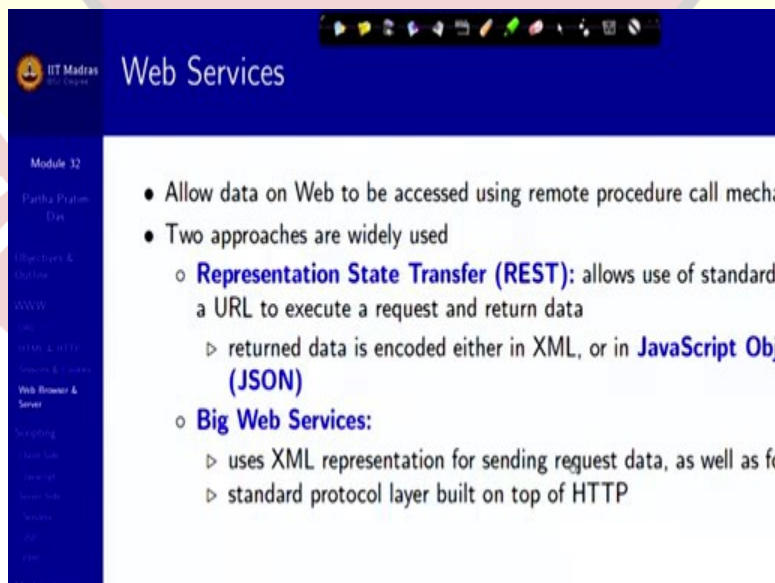


Web Servers

- A web server is software and underlying hardware that accepts requests and serves secure variant HTTPS
- A web browser or crawler, requests for a specific resource using HTTP and responds with the content of that resource or an error message
- The server can also accept and store resources sent from the user agent
- The document name in a URL may identify an executable program, and the server generates a HTML document
 - When an HTTP server receives a request for such a document, it executes the program, and sends back the HTML document that is generated
 - The Web client can pass extra arguments with the name of the document
- To install a new service on the Web, one simply needs to create and run an executable that provides that service

Web server on the other side is a basic service provider, which gets a HTTP request, forms a response to that for some by through some mechanism, makes it into an HTML and sends you back. So, it could be a web service, could be simply another program which does this or it could be a big web server and as you as you learn more you will see that. But the basic purpose of the web server is to get an HTTP request and give out the result HTML.

(Refer Slide Time: 17:57)



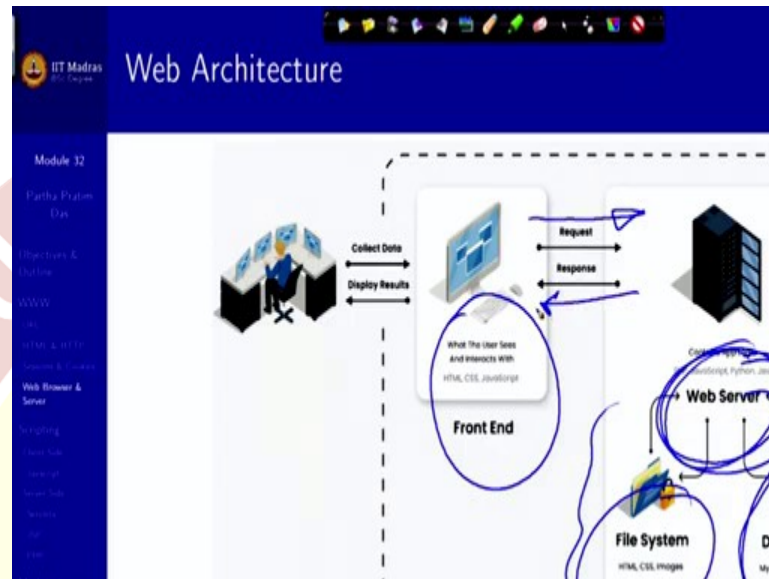
Web Services

- Allow data on Web to be accessed using remote procedure call mechanism
- Two approaches are widely used
 - **Representation State Transfer (REST)**: allows use of standard HTTP and a URL to execute a request and return data
 - ▷ returned data is encoded either in XML, or in **JavaScript Object Notation (JSON)**
 - **Big Web Services**:
 - ▷ uses XML representation for sending request data, as well as for response
 - ▷ standard protocol layer built on top of HTTP

Now, other than HTTP also, there are other kinds of services particularly used for transferring larger volumes of data or maybe some unstructured data they are known as REST, JSON and so

on. So, when you learn about web services in depth, you will learn about them. I just wanted to mention so that you know where these terms exist.

(Refer Slide Time: 18:20)



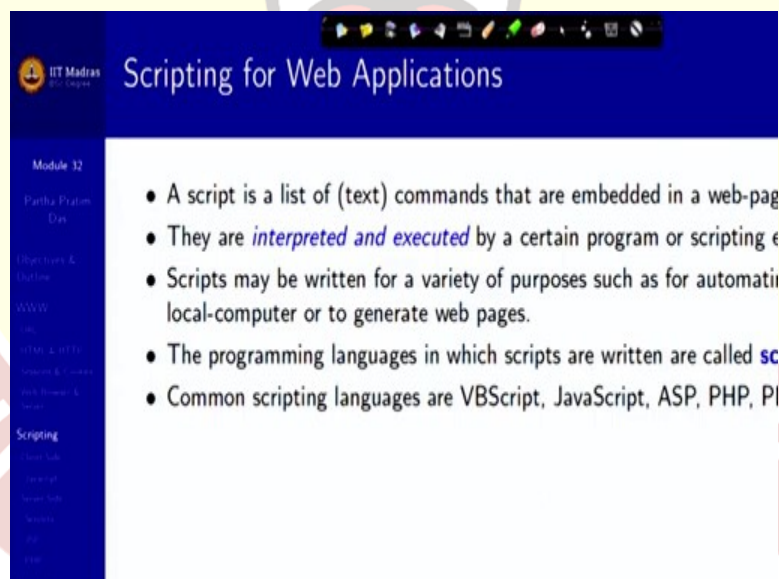
So, this is, in short, this is what, where we are. So, this is in your web architecture, this is your frontend, this is your web server, which will actually go to the database or the file system. It could get the document directly from the file system, it could run a query from the database, get it here, make it into HTML. So, you send a request here, which goes in HTTP, the web server does this backend job and sends back a response which is HTML. So, this is a basic web architecture and you can see that the application architecture is basically trying to follow that.

(Refer Slide Time: 18:56)



Now, we will briefly talk about scripting.

(Refer Slide Time: 19:00)



Scripting, scripts are nothing but programming languages, but the difference being that they are typically text, they are small, and they are usually interpreted within another program. So, typically, when we write a C++ program, we need to first write the program, compile it, and on compilation correct, we execute. So, if we make a change in the source program, my executable does not change, but the script is not like that. Script is a text, piece of text, which is a program, some instructions, and every time you want to run it, you need to interpret that is you compile

and run at the same time. These are typically small and done for dynamic systems. So, there are several scripting languages. Some of them I have mentioned here.

(Refer Slide Time: 19:51)

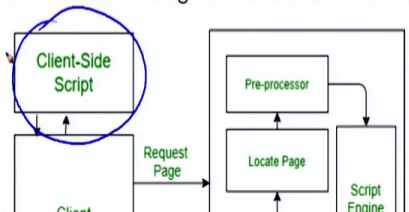
**IIT Madras**
BSc - Computer Science

Scripting for Web Applications (2)

Module 32
Partha Pratim Das
Objectives & Outline
WWW
URL
HTML & HTTP
Sessions & Cookies
Web Browser & Server
Scripting
Client Side
JavaScript
Server Side
Servlets
JSP
PHP

Scripting of two types

- **Client Side:** Client-side scripting is responsible for interaction within client-side scripts are firstly downloaded at the client-end and then in executed by the browser
- **Server Side:** Server-side scripting is responsible for the completion o task at the server-end and then sending the result to the client-end.



```
graph LR; Client[Client] -- "Request Page" --> LocatePage[Locate Page]; LocatePage --> PreProcessor[Pre-processor]; PreProcessor --> ScriptEngine[Script Engine]; ScriptEngine --> ClientSideScript[Client-Side Script]; ClientSideScript --> Client;
```

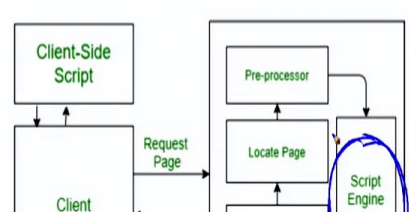
**IIT Madras**
BSc - Computer Science

Scripting for Web Applications (2)

Module 32
Partha Pratim Das
Objectives & Outline
WWW
URL
HTML & HTTP
Sessions & Cookies
Web Browser & Server
Scripting
Client Side
JavaScript
Server Side
Servlets
JSP
PHP

Scripting of two types

- **Client Side:** Client-side scripting is responsible for interaction within client-side scripts are firstly downloaded at the client-end and then in executed by the browser
- **Server Side:** Server-side scripting is responsible for the completion o task at the server-end and then sending the result to the client-end.



```
graph LR; Client[Client] -- "Request Page" --> LocatePage[Locate Page]; LocatePage --> PreProcessor[Pre-processor]; PreProcessor --> ScriptEngine[Script Engine]; ScriptEngine --> ClientSideScript[Client-Side Script]; ClientSideScript --> Client;
```




IIT Madras
MADRAS INSTITUTE OF TECHNOLOGY

Module 32

Partha Pratim Das

Objectives & Outline

WWW

URL

HTML & HTTP

Sessions & Cookies

Web Browser & Server

Scripting

Client Side

Server Side

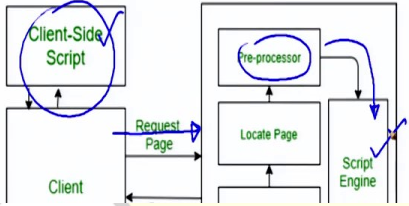
PHP

Python

Scripting for Web Applications (2)

Scripting of two types

- **Client Side:** Client-side scripting is responsible for interaction within client-side scripts are firstly downloaded at the client-end and then in executed by the browser
- **Server Side:** Server-side scripting is responsible for the completion o task at the server-end and then sending the result to the client-end.



Now, here, there are a lot of tasks for which we need scripting in the in case of web application. Many of those scripts run in the browser and they are called client-side script. For example, I put a date and it needs to be validated to be corrected. You could have done it through going to the server also. You could take the date, go to the server, server or application layer checks, whether it is a valid date comes back and tells you. That would mean a lot more of Internet time, that would mean lot of delay.

Whereas to check for a valid date is a simple logic. So, you can implement it in terms of a script, which can run within the browser. This is an interesting concept that you are running a program not directly on the machine, but you are running it through a simulation and that is in the browser. So, it is something like you have a similar situation for a JVM or PVM for Java and Python, but this is a very typical client-side scripting that you can do within the browser itself.

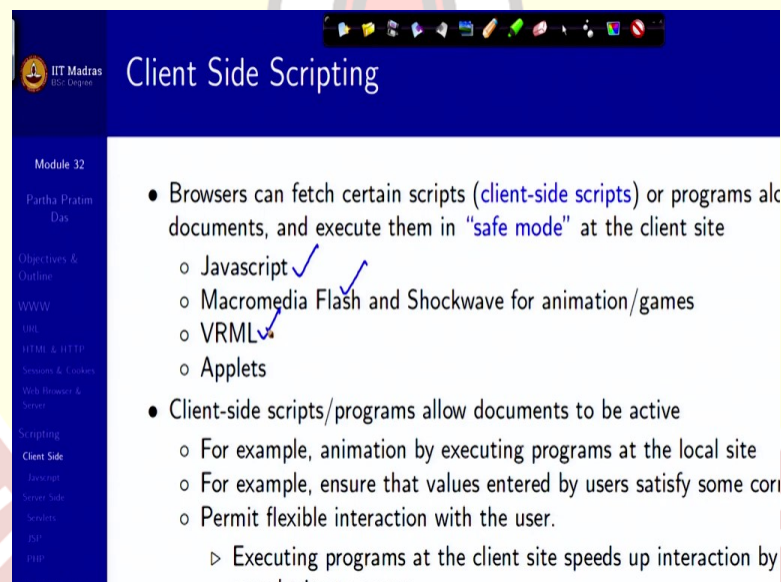
The other option is you will have several such requirements of processing, validation, extraction in the server-end. So, those are also often done in terms of scripting language. Those are called clients, server-side scripting. So, you have scripting, but these two are different. At the server side, you could be doing connecting to a database, you could be extracting data and so on, so forth. Now, the question is, why do you use it as a script, because many times every individual task is a small one. And it is after having done the task you need to create the HTML.

So, this process of creating the HTML or the tags and so on, so that you can display it nicely, is far more difficult to do in a normal programming language than in a script which you can

visualize as if you have an HTML page with certain information missing, the script can come and set those information so that your page is ready. Now, since this is in the server, this has lot more power. This can have a lot more computing. So, typically, it does not, unlike the client side, it does not run within another application, you may have a separate engine which runs your script, which is called the scripting engine.

So, a modified view turns out to be that you have client-side scripts which validates. And when it is validated, then the request can come, it is taken up by the preprocessor, chooses the right script in the scripting engine, which does the scripting, does the computation, gets the data, generates the result as a translated HTML, locates that page template, gets the final HTML, returns it back here to the client. So, scripting at both these two ends basically are technological mechanisms to make the whole system far more efficient and easy to use and easy to change.

(Refer Slide Time: 23:21)

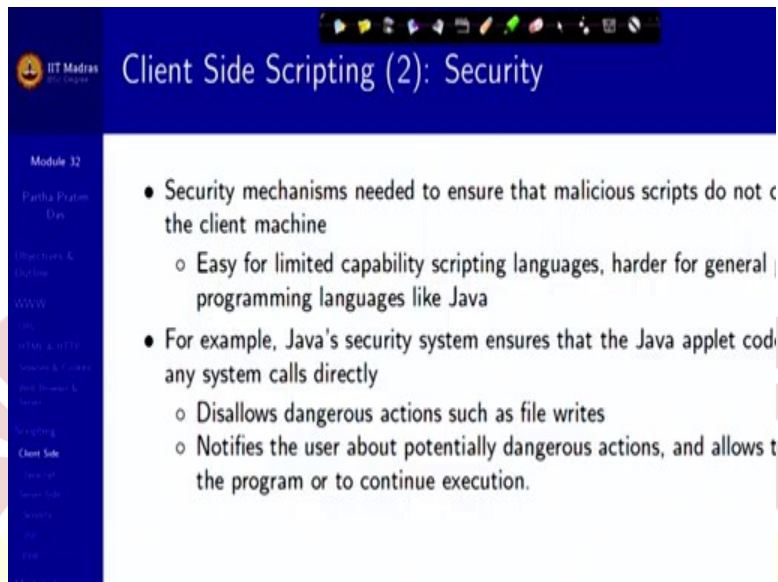


The screenshot shows a presentation slide titled "Client Side Scripting" from IIT Madras. The slide is part of Module 32, presented by Partha Pratim Das. The left sidebar lists the following topics: Objectives & Outline, WWW, URL, HTML & HTTP, Sessions & Cookies, Web Browser & Server, Scripting, Client Side, Javascript, Server Side, Security, PHP, and JSP. The main content area lists the following points:

- Browsers can fetch certain scripts (**client-side scripts**) or programs along with documents, and execute them in "safe mode" at the client site
 - Javascript ✓
 - Macromedia Flash and Shockwave for animation/games
 - VRML ✓
 - Applets
- Client-side scripts/programs allow documents to be active
 - For example, animation by executing programs at the local site
 - For example, ensure that values entered by users satisfy some constraints
 - Permit flexible interaction with the user.
 - ▷ Executing programs at the client site speeds up interaction by reducing the need to communicate with the server

So, there are several ways to do client-side scripting. Many browsers execute them in the safe mode at the client side, which means that your security is not compromised. There are several naturally JavaScript you must have heard of. This is something you need a flash, plug-in to be set to see different media files. This is to see your SVG kind of dynamic graphics files, applets and so on. It could be variety of different these kinds of client side scripting options exist.

(Refer Slide Time: 24:04)

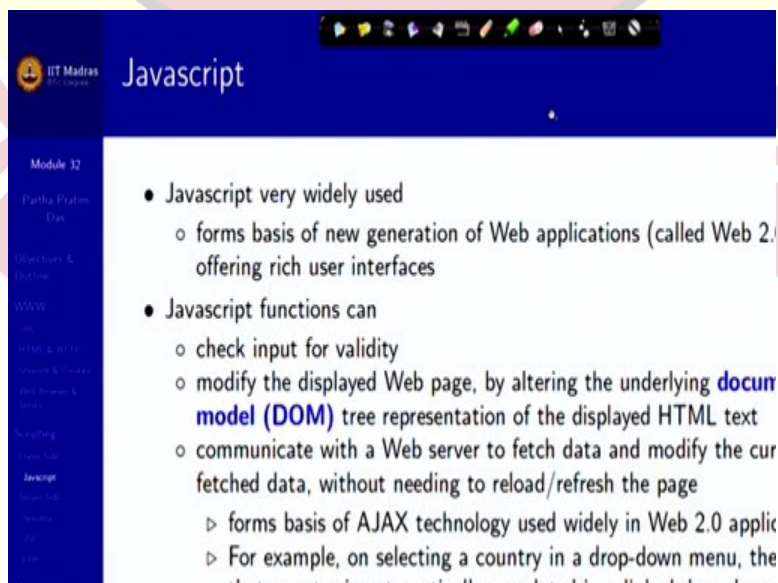


Client Side Scripting (2): Security

- Security mechanisms needed to ensure that malicious scripts do not compromise the client machine
 - Easy for limited capability scripting languages, harder for general programming languages like Java
- For example, Java's security system ensures that the Java applet code does not make any system calls directly
 - Disallows dangerous actions such as file writes
 - Notifies the user about potentially dangerous actions, and allows the user to either deny the program or to continue execution.

Now, security is certainly a concern for the client-side scripting, because what if some malicious script has come to your machine, and through the browser, it spoils your machine. So, for that, there are very limited options given to the scripting language at the client side. For example, typically, the scripts are not allowed to write anything on your disk, because then they could write a virus. It will not allow any Java applet code or equivalent scripting language will not allow anything to be executed outside of the browser. So, that security is kind of built in.

(Refer Slide Time: 24:45)



Javascript

- Javascript very widely used
 - forms basis of new generation of Web applications (called Web 2.0) offering rich user interfaces
- Javascript functions can
 - check input for validity
 - modify the displayed Web page, by altering the underlying **document model (DOM)** tree representation of the displayed HTML text
 - communicate with a Web server to fetch data and modify the current fetched data, without needing to reload/refresh the page
 - ▷ forms basis of AJAX technology used widely in Web 2.0 applications
 - ▷ For example, on selecting a country in a drop-down menu, the data is fetched from a server and displayed in a text box without reloading the page

The most common is JavaScript which follows a document object model. I will not get into the details of that. You study when you do the web services in general.

(Refer Slide Time: 24:58)

Module 12
Partha Pratim Das
Higher Level & Outflow
Web
HTML & CSS
JavaScript
AJAX
JSON
XML
SOAP
REST
Web Services
Security
Cloud Computing
Big Data
IoT
Blockchain
AI/ML
AR/VR
Quantum Computing
Cybersecurity
Ethical Hacking
DevOps
Cloud Native
Serverless
Edge Computing
5G
Quantum Internet
Space Technology
Autonomous Systems
Bio-Inspired Computing
Neuromorphic Computing
Quantum Cryptography
Blockchain
AI/ML
AR/VR
Quantum Computing
Cybersecurity
Ethical Hacking
DevOps
Cloud Native
Serverless
Edge Computing
5G
Quantum Internet
Space Technology
Autonomous Systems
Bio-Inspired Computing
Neuromorphic Computing
Quantum Cryptography

Javascript (2): Example

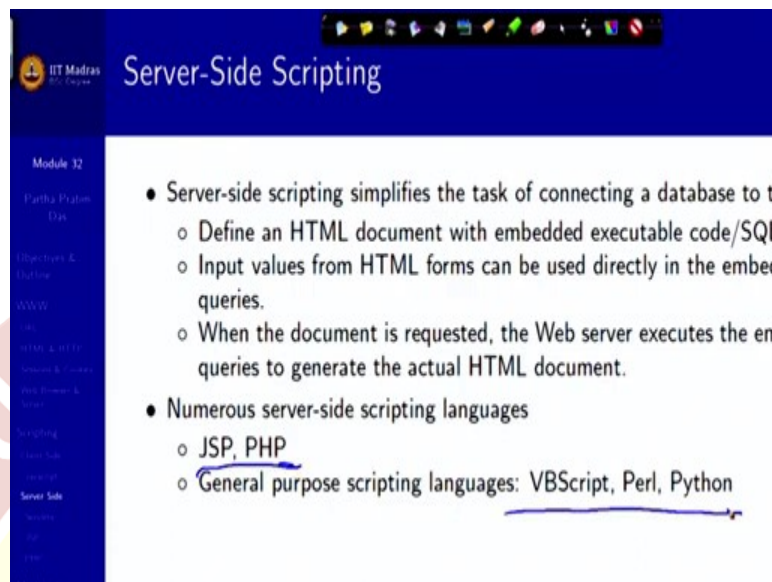
- Example of Javascript used to validate form input

```
<html> <head>  
  <script type="text/javascript">  
    function validate() {  
      var credits=document.getElementById("credits").value;  
      if (isNaN(credits) || credits<=0 || credits>=16) {  
        alert("Credits must be a number greater than 0 and 16");  
        return false;  
      }  
    }  
  </script>  
</head> <body>  
  <form action="createCourse" onsubmit="return validate()">  
    Title: <input type="text" id="title" size="20"><br />  
    Credits: <input type="text" id="credits" size="20"><br />  
    <input type="button" value="Create Course" />  
  </form>  
</body>  
</html>
```

But this is just to give you an example. For example, here, you say that the script is JavaScript and you are doing a validation function. What it is saying that you have taken a credit amount, credits amount by getting the element by ID. So, credits was a HTML element, some value has been entered and you take that. And what you are checking? You checking is NAN, NAN is not a number. The credit has to be a number. So, if it is not a number, it fails. This becomes true that is it is not a number.

Is credit less than equal to 0 or if credit greater than equal to 16. If that happens, then you throw an alert. So, what you get to see that you enter 5, it works okay. You enter -3 or you enter PPD or you enter 17, you will get an alert, saying that the credit must be and this whole thing is happening without an HTTP request being sent to the sender all in the JavaScript. So, that is the basic mechanism of JavaScript.

(Refer Slide Time: 26:13)

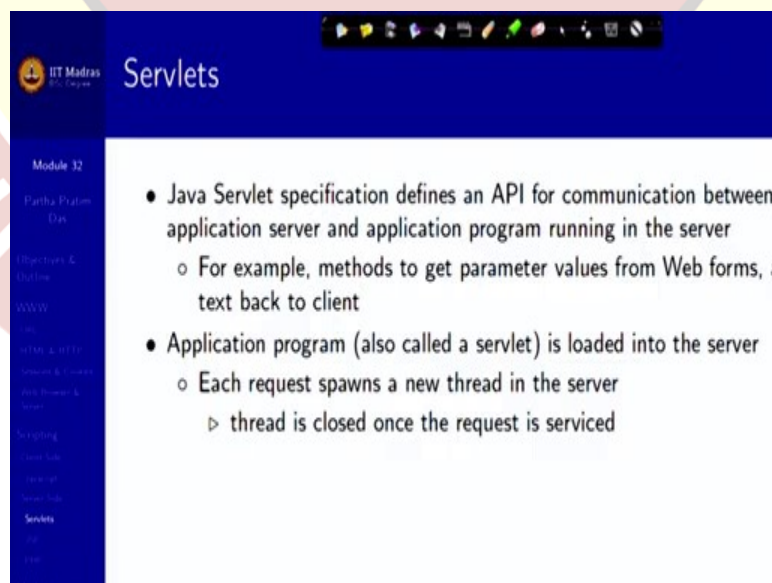


Server-Side Scripting

- Server-side scripting simplifies the task of connecting a database to the client
 - Define an HTML document with embedded executable code/SQL
 - Input values from HTML forms can be used directly in the embedded queries.
 - When the document is requested, the Web server executes the embedded queries to generate the actual HTML document.
- Numerous server-side scripting languages
 - JSP, PHP
 - General purpose scripting languages: VBScript, Perl, Python

Similarly, you have server-side scripts, as I said, which are, which take a request, do the scripting, processing and prepares your final HTML for return. The most commonly used are these days are JSP, PHP once ASP used to be also very popular earlier. But some general-purpose languages like Python can also be used for this purpose because they are also scripting languages.

(Refer Slide Time: 26:46)



Servlets

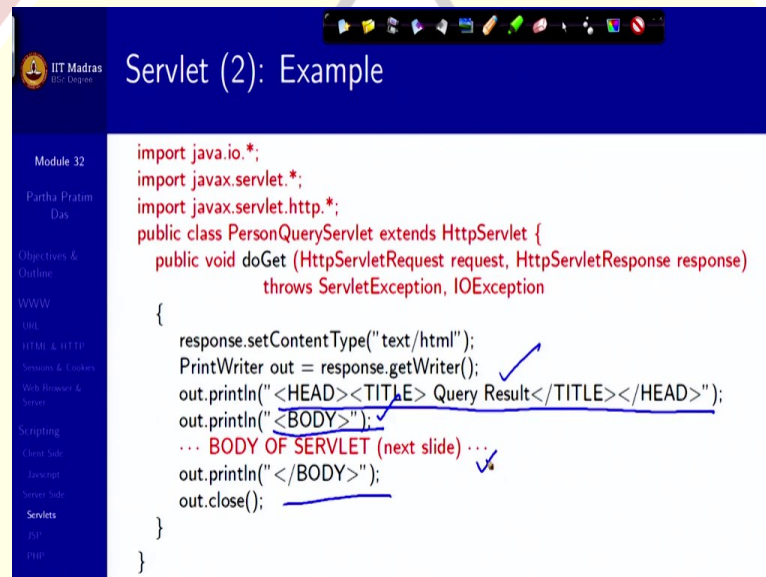
- Java Servlet specification defines an API for communication between application server and application program running in the server
 - For example, methods to get parameter values from Web forms, and text back to client
- Application program (also called a servlet) is loaded into the server
 - Each request spawns a new thread in the server
 - ▷ thread is closed once the request is serviced

And as a part of the scripting, as I said, that instead of just running the script within another program, it makes it into a small program by itself. So, these are called servlets. Why servlets?

They are servers, because they are providing a service based on the input. And they are like book to booklet, is a small book similarly server to servlet is a small service, just one small service, and you can have many of them.

So, that it becomes a lot more manageable. You do not, because at a time at any request, you do not have too many things to do. Every request is a small thing and you would have possibly a separate servlet. They run on thread, because same instances of the, I mean, multiple instances of the same servlet could run at the same time, because different users have requested for the same service. So, servlet is a very strong concept in the on the server side.

(Refer Slide Time: 27:54)



```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class PersonQueryServlet extends HttpServlet {
    public void doGet (HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException
    {
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("<HEAD><TITLE> Query Result</TITLE></HEAD>");
        out.println("<BODY>");
        ... BODY OF SERVLET (next slide) ...
        out.println("</BODY>");
        out.close();
    }
}
```

So, these are some typical examples which is running the servlet and, in the format, actually filling in the HTML. So, this is a header line, this is a body part we had shown and what does it get in the body you can.

(Refer Slide Time: 28:19)

```
String persontype = request.getParameter("persontype");
String number = request.getParameter("name");
if (persontype.equals("student")) {
    ... code to find students with the specified name ...
    ... using JDBC to communicate with the database ...
    out.println("<table BORDER COLS=3>");
    out.println("<tr> <td>ID</td><td>Name</td><td>Department</td></tr>");
    for (... each result ...) {
        ... retrieve ID, name and dept name
        ... into variables ID, name and deptname
        ... out.println("<tr> <td>" + ID + "</td>" + "<td>" + name
        ... + "</td>" + "<td>" + deptname + "</td></tr>");
    }
    out.println("</table>");
}
```

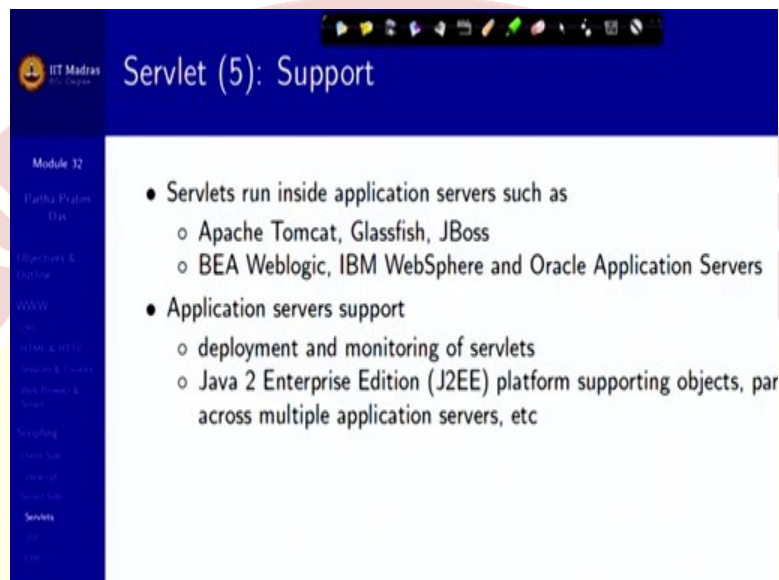
This is where it is building the body. It goes to the database to get the data. The HTMLs for the table and this is for the header part we had seen and this is where one by one for each as many results have come for each it is creating a row and putting in, very simple. I mean, the HTML example you saw that is getting automatically generated based on the result that we have got from the database. How we got it from the database is something we will discuss, but this is how a servlet typically would work.

(Refer Slide Time: 28:58)

- Servlet API supports handling of sessions
 - Sets a cookie on first interaction with browser, and uses it to identify further interactions
- To check if session is already active:
 - if (request.getSession(false) == true)
 - ▷ .. then existing session
 - ▷ else .. redirect to authentication page
 - authentication page
 - ▷ check login/password
 - ▷ request.getSession(true): creates new session
- Store/retrieve attribute value pairs for a particular session

So, naturally, servlets have sessions. We have talked about. So, you can see that for a request if the get session tells you that the session is live then you continue, otherwise you redirect to authenticate and create, start a new session after authentication and keep that as a new cookie.

(Refer Slide Time: 29:23)

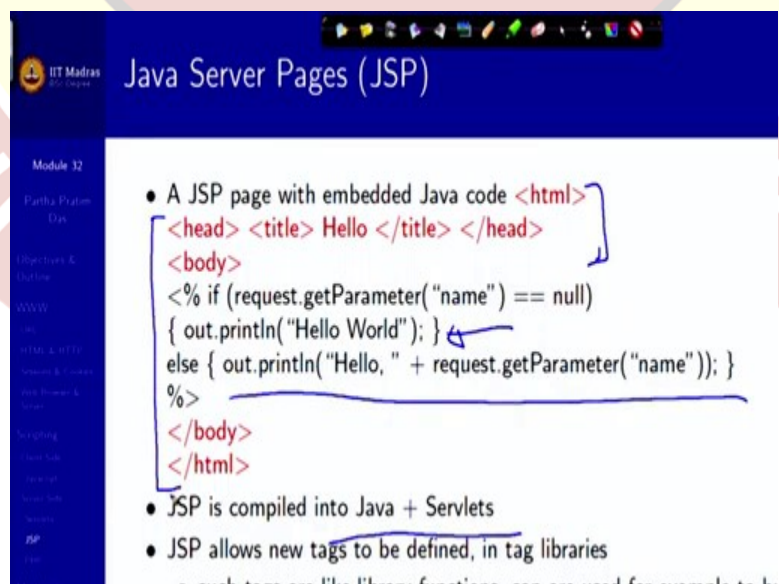


Servlet (5): Support

- Servlets run inside application servers such as
 - Apache Tomcat, Glassfish, JBoss
 - BEA Weblogic, IBM WebSphere and Oracle Application Servers
- Application servers support
 - deployment and monitoring of servlets
 - Java 2 Enterprise Edition (J2EE) platform supporting objects, par across multiple application servers, etc

This is servlets are supported on multiple web servers like Apache Tomcat is one of the most widely used then IBM, WebSphere, there are several of them. J2EE has options for that.

(Refer Slide Time: 29:38)



Java Server Pages (JSP)

- A JSP page with embedded Java code `<html>`

```
<head> <title> Hello </title> </head>
<body>
<% if (request.getParameter("name") == null)
{ out.println("Hello World"); }
else { out.println("Hello, " + request.getParameter("name")); }
%>
</body>
</html>
```
- JSP is compiled into Java + Servlets
- JSP allows new tags to be defined, in tag libraries
 - such tags are like library functions, are used for example to b

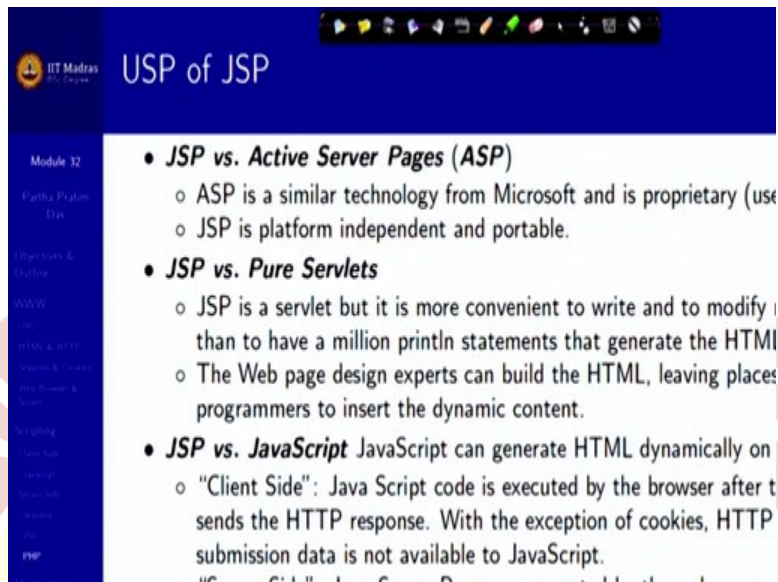
The other one, which is very popular that is java server page, which is kind of the same concept, the only difference being that there you are using it like a programming language and generating the HTML. Here you have the HTML in the format and you are just putting the, so you have written as if an HTML and then you have certain parts which are missing, which is like what you will put in this, hello, the name part we will get as a parameter and put that so that part is missing. So, it is compiled into java plus servlet and then you run it. The advantage is it is much easier to write an HTML embedding in this way, instead of writing it by text output as we are seeing here earlier.

(Refer Slide Time: 30:30)



Similarly, you have PHP and other mechanism at the server scripting.

(Refer Slide Time: 30:36)

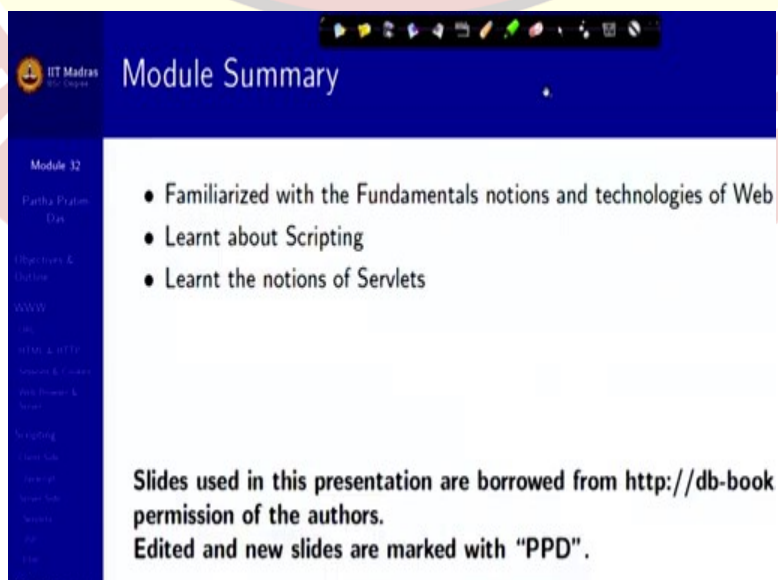


USP of JSP

- **JSP vs. Active Server Pages (ASP)**
 - ASP is a similar technology from Microsoft and is proprietary (use)
 - JSP is platform independent and portable.
- **JSP vs. Pure Servlets**
 - JSP is a servlet but it is more convenient to write and to modify than to have a million `println` statements that generate the HTML
 - The Web page design experts can build the HTML, leaving places programmers to insert the dynamic content.
- **JSP vs. JavaScript** JavaScript can generate HTML dynamically on
 - "Client Side": Java Script code is executed by the browser after it sends the HTTP response. With the exception of cookies, HTTP submission data is not available to JavaScript.

And JSP is actually getting preferred a lot over the competitive ones like ASP. ASP is proprietary for Microsoft and uses proprietary VB, JSP is platform independent, portable easily. Against pure servlets, as I said, it is much easier to write the HTML format of JSP than having to write them by `cout` statements or `print line` statements and so on. And certainly, do not confuse between JSP Java Server Page and JavaScript. JavaScript is client side, runs in the browser. JSP server side, runs in the server, and obviously it is much better than static HTML which cannot have a dynamic information.

(Refer Slide Time: 31:24)



Module Summary

- Familiarized with the Fundamentals notions and technologies of Web
- Learnt about Scripting
- Learnt the notions of Servlets

Slides used in this presentation are borrowed from <http://db-book> permission of the authors.
Edited and new slides are marked with "PPD".

So, we have got introduced to some basic notions of the web fundamentals and the technology for applications about including scripting and servlets. This brings us to the end of the module. See you again in the next module. Thank you for your attention. We will continue our discussions on the application development.

